

Q238 | LC 1291 | Sequential Digits

© Authored by: Arpit Raj, LNMIIT Jaipur | Date: July 14, 2026

Apple, Meta

Problem Outline

- -> between **l** and **h** we need to return all numbers which are sequential
- Eg [100, 300] -> sequential number in this range are 123, 234

2 Approaches

Approach 1 .. (Enumeration)

- there are very few sequential digits
- Eg [100, 300] both are 3 digit numbers so we need a 3 digit sequential number
- to get sequential number we can use the string ["123456789"] and check which all them lies in range

TC(O(1)) | SC(O(1))

```
class Solution {
public:
    vector<int> sequentialDigits(int low, int high) {
        string lookup="123456789";
        vector<int> ans;
        int llen=to_string(low).size();
        int hlen=to_string(high).size();

        for(int len=llen;len<=hlen;len++){
            for(int i=0;i+len<=9;i++){

                int num=stoi(lookup.substr(i,len));
                if(num>=low && num<=high){
                    ans.push_back(num);
                }

                else if(num>high){
                    return ans;
                }
            }
        }
        return ans;
    }
};
```

Approach 2 BFS

- every sequential number extends its previous digit
 - Eg [1 -> 12 -> 123 -> 1234]
 - So **newnum = num * 10 + (last digit + 1)**
 - hence check for all 1, 2, 3, ... 9
 - make each of them append numbers, and if the number is in range push to ans vector
 - So **currnum** in [l, h], push to ans
 - **nextnum less than h**, push to queue (we can make use of it to generate more number)
-

Example Flow: Eg [100, 300] -> 1,2,3,4,5,6,7,8,9 -> 12

2,3,4,5,6,7,8,9,12

similarly (12, 23, 34, 45, 56, 67, 78, 89)

and then (123, 234, 345, 456, 567, 678, 789)

in range and less than so push to ans and 1234 push to q

TC = $O(1)$ | SC = $O(1)$

```
class Solution {
public:
    vector<int> sequentialDigits(int low, int high) {

        queue<int> q;
        vector<int> ans;
        for (int i = 1; i <= 9; i++) {
            q.push(i);
        }

        while (!q.empty()) {
            int num = q.front();
            q.pop();

            if (num >= low && num <= high) {
                ans.push_back(num);
            }

            int last = num % 10;
            if (last != 9) {
                int newnum = (num * 10) + last + 1;
                if (newnum <= high) {
                    q.push(newnum);
                }
            }
        }

        return ans;
    }
};
```